JsonPropertyOrder Explanation

whats JsonPropertyOrder

`@JsonPropertyOrder` is a **Jackson annotation** used to control the order of fields when a Java object is serialized to JSON.

Without `@JsonPropertyOrder`

```Java
public class Employee {
    private int id;
    private String name;
    private String department;
}
```

Output order may be:

```JSON
{
  "department": "IT",
  "id": 1,
  "name": "Sumit"
}
```

Jackson doesn't guarantee a specific order unless configured.

With `@JsonPropertyOrder`

```Java
import com.fasterxml.jackson.annotation.JsonPropertyOrder;
@JsonPropertyOrder({"id", "name", "department"})
public class Employee {
    private int id;
    private String name;
    private String department;
}
```

Output:

```JSON
{
  "id": 1,
  "name": "Sumit",
  "department": "IT"
}
```

Alphabetical Order

You can also ask Jackson to sort fields alphabetically:

```Java
@JsonPropertyOrder(alphabetic = true)
public class Employee {
    private String department;
    private int id;
    private String name;
}
```

Output:

```JSON
{
  "department": "IT",
  "id": 1,
  "name": "Sumit"
}
```

Common Use Cases

1. API Contract Consistency

If another system expects fields in a specific order.

2. Readable Logs/Reports

Useful when JSON is printed in logs or Extent Reports.

3. Snapshot Testing

When comparing JSON strings directly, a fixed order prevents false test failures.

Example with POJO used in RestAssured

```Java
@JsonPropertyOrder({"name", "age", "city"})
public class User {

    private String name;
    private int age;
    private String city;

    // getters/setters
}
```

```Java
User user = new User();
user.setName("Sumit");
user.setAge(39);
user.setCity("Pune");

ObjectMapper mapper = new ObjectMapper();

String json = mapper.writeValueAsString(user);
System.out.println(json);
```

Output:

```JSON
{
  "name":"Sumit",
  "age":39,
  "city":"Pune"
}
```

Interview Question

Does `@JsonPropertyOrder` affect deserialization?

No. It only controls the order when converting Java objects → JSON. When reading JSON → Java object, field order doesn't matter.

and if its nested object, u need that on sub object also?

Yes.

`@JsonPropertyOrder` applies **only to the class on which it is declared**. If you have nested objects and you want a specific field order inside those nested objects, you need to annotate those classes separately.

Example

```Java
@JsonPropertyOrder({"id", "name", "address"})
public class Employee {
    private int id;
    private String name;
    private Address address;
}
```

```Java
@JsonPropertyOrder({"city", "state", "country"})
public class Address {
    private String country;
    private String city;
    private String state;
}
```

Object:

```Java
Employee emp = new Employee(
    1,
    "Sumit",
    new Address("India", "Pune", "Maharashtra")
);
```

Output:

```JSON
{
  "id": 1,
  "name": "Sumit",
  "address": {
    "city": "Pune",
    "state": "Maharashtra",
    "country": "India"
  }
}
```

Notice:

- **`Employee`** fields follow **`id → name → address`**
- **`Address`** fields follow **`city → state → country`**

because both classes have their own **`@JsonPropertyOrder`**.

If you **don't** put **`@JsonPropertyOrder`** on **`Address`**:

```Java
@JsonPropertyOrder({"id", "name", "address"})
public class Employee {
    ...
}
```

then only the top-level order is guaranteed:

```JSON
{
  "id": 1,
  "name": "Sumit",
  "address": {
    "country": "India",
    "city": "Pune",
    "state": "Maharashtra"
  }
}
```

The order inside **`address`** is then determined by Jackson's default behavior.

Rule of thumb

Think of **`@JsonPropertyOrder`** as a **class-level setting**:

- Want order in **`Employee`** → annotate **`Employee`**
- Want order in **`Address`** → annotate **`Address`**
- Want order in **`Department`** → annotate **`Department`**

Each POJO controls its own JSON field order.